

OOP

Inheritance

Polymorphism

Abstraction



Build on Session 2



Exercises



OOP Project

Prerequisite: Constructors, Encapsulation, Overloading, Static (Session 2)



Session Roadmap

01

Inheritance

~20 min

extends · super() · method overriding · @Override

03

Polymorphism

~20 min

Runtime dispatch · instanceof · casting · arrays of base

05

Interfaces

~15 min

interface · implements · default methods · multiple impl

02

super Keyword

~15 min

Field access · constructor chaining · method delegation

04

Abstract Classes

~15 min

abstract class · abstract method · template method pattern

06

Mini Project

~35 min

Animal Kingdom — full OOP hierarchy

01 Inheritance — Extending Existing Classes



What is Inheritance?

Inheritance lets a child class reuse fields and methods from a parent class using the extends keyword. The child gets everything the parent has (except private members) and can add its own.

Key Rules

- Java supports single inheritance (one parent only)
- Use extends keyword in the class declaration
- Child can access public and protected members
- Child CANNOT access private members directly
- Every class implicitly extends Object

```
// Animal.java — parent class
public class Animal {
    String name;
    int age;

    Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void eat() {
        System.out.println(name + " is eating");
    }
    void sleep() {
        System.out.println(name + " is sleeping");
    }
}

// Dog.java — child class
public class Dog extends Animal {
    String breed;

    Dog(String name, int age, String breed) {
        super(name, age); // call parent constructor
        this.breed = breed;
    }

    void bark() {
        System.out.println(name + " says: Woof!");
    }
}
```

```
Dog dog = new Dog("Rex", 3, "Labrador"); dog.eat(); dog.sleep(); dog.bark(); // all work!
```

Dog inherits eat() and sleep() from Animal — no need to rewrite them!

01 Inheritance — Method Overriding & @Override



What is Overriding?

A child class can provide its own implementation of a method already defined in the parent. The new version must have the **SAME** signature (name + parameters + return type).

Always use `@Override` — it tells the compiler you intend to override, catching typos early.

Overriding vs Overloading

	Overriding	Overloading
Where	Child class	Same class
Signature	Must match	Must differ
When	Runtime	Compile time
@Override	Use it	Not applicable

```
// Animal.java
public class Animal {
    String name;
    Animal(String name) { this.name = name; }

    void makeSound() {
        System.out.println(name + " makes a sound");
    }
}

// Dog.java
public class Dog extends Animal {
    Dog(String name) { super(name); }

    @Override
    void makeSound() {
        System.out.println(name + " says: Woof!");
    }
}

// Cat.java
public class Cat extends Animal {
    Cat(String name) { super(name); }

    @Override
    void makeSound() {
        System.out.println(name + " says: Meow!");
    }
}

// Main
Animal a = new Dog("Rex");
a.makeSound(); // → Rex says: Woof!
```

`@Override` catches bugs: if you misspell the method name, the compiler reports an error instead of silently creating a new method.



Exercise 1 — Inheritance & Overriding



Task: Vehicle Hierarchy

- 1 Create a Vehicle parent class with fields: String brand, int speed, int year. Add a constructor and a describe() method.
- 2 Create Car extends Vehicle — adds int doors. Override describe() to include doors.
- 3 Create Truck extends Vehicle — adds double cargoTons. Override describe() to include cargo.
- 4 Create Motorcycle extends Vehicle — adds boolean hasSidecar. Override describe().
- 5 In main, create one of each. Call describe() on all three.
- 6 **EDGE CASE:** Create an array Vehicle[] vehicles = {car, truck, moto}; and loop to call describe() on each.

► Expected Output

```
Car: Toyota | 2022 | Speed: 180 | Doors: 4
Truck: Volvo | 2020 | Speed: 120 | Cargo: 8.5t
Moto: Honda | 2023 | Speed: 200 | Sidecar:
false
-- Loop output --
[same three lines repeated]
```



Hint — super.describe()

```
@Override
void describe() {
    super.describe(); // reuse parent
    System.out.println("Doors: " + doors);
}
```



Time: 18 minutes

02 The super Keyword

① Access parent fields

When a child field shadows a parent field with the same name, use `super.fieldName` to reach the parent's version.

```
class Child extends Parent {
    String name = "Child";
    void show() {
        System.out.println(super.name); // Parent's
        System.out.println(this.name); // Child's
    }
}
```

② Call parent constructor

`super(...)` in the first line of a child constructor calls the parent constructor. If omitted, Java inserts `super()` automatically — which fails if the parent has no no-arg constructor.

```
class Dog extends Animal {
    Dog(String name, int age) {
        super(name, age); // MUST be first line
        // ... Dog-specific init ...
    }
}
```

③ Call parent method

After overriding a method, you can still invoke the parent's version inside the override using `super.methodName()`.

```
class Dog extends Animal {
    @Override
    void describe() {
        super.describe(); // reuse parent logic
        System.out.println("Breed: " + breed);
    }
}
```

 `super()` must always be the FIRST statement in a constructor — same rule as `this()`.

03 Polymorphism — One Reference, Many Forms

Polymorphism means you can hold a child object in a parent-type variable. When you call an overridden method, Java decides at runtime which version to run — based on the actual object, not the variable type. This is called dynamic dispatch.

```
Animal a1 = new Dog("Rex");    // Dog stored as Animal
Animal a2 = new Cat("Whiskers");
Animal a3 = new Animal("Generic");

a1.makeSound(); // → Rex says: Woof!    (Dog's version)
a2.makeSound(); // → Whiskers says: Meow! (Cat's version)
a3.makeSound(); // → Generic makes a sound (Animal's)

// Polymorphic array – very powerful!
Animal[] zoo = { new Dog("Rex"), new Cat("Luna"),
                 new Parrot("Polly") };
for (Animal a : zoo) {
    a.makeSound(); // each runs its own override
}
```

instanceof & Casting

Check the actual type before casting to avoid `ClassCastException`:

```
Animal a = new Dog("Rex");

if (a instanceof Dog) {
    Dog d = (Dog) a; // safe cast
    d.bark(); // Dog-only method
}

// Pattern (Java 16+):
if (a instanceof Dog d) {
    d.bark();
}
```



Key insight: write code against the parent type — it works for ALL current and future child classes without changes.



Exercise 2 — Polymorphism in Action



Task: Shape Area Calculator

- 1 Create a Shape parent class with a String color field and a double area() method that returns 0.
- 2 Create Circle extends Shape — adds double radius. Override area() using $\text{Math.PI} * r * r$.
- 3 Create Rectangle extends Shape — adds double width, height. Override area().
- 4 Create Triangle extends Shape — adds double base, height. Override area() ($\text{base} * \text{height} / 2$).
- 5 In main, build a Shape[] with one of each and loop to print color + area for every shape.
- 6 **EDGE CASE:** Add a method printIfBig(Shape s, double threshold) — uses instanceof to also print the shape type name.

► Expected Output

```
Red Circle: area = 78.54
Blue Rectangle: area = 24.0
Green Triangle: area = 15.0

-- printIfBig (threshold 20.0) --
Red Circle [Circle] area=78.54
Blue Rectangle [Rectangle] area=24.0
```



Hint

```
void printIfBig(Shape s, double t) {
    if (s.area() > t) {
        String type = "Unknown";
        if (s instanceof Circle) type="Circle";
        // ... etc
        System.out.printf("%s [%s] area=%.2f\n",
            s.color, type, s.area());
    }
}
```



Time: 18 minutes

04 Abstract Classes — Enforcing a Contract



What is abstract?

An abstract class cannot be instantiated directly. It can have both concrete methods (with a body) and abstract methods (no body). Child classes MUST implement every abstract method.

- Cannot create objects: `new Shape()` → error
- Can have constructors (called via `super()`)
- Can have concrete helper methods (shared logic)
- Child must override ALL abstract methods (or be abstract itself)

```
// Circle.java - concrete child
public class Circle extends Shape {
    double radius;
    Circle(String color, double radius) { super(color); this.radius = radius; }

    @Override double area()          { return Math.PI * radius * radius; }
    @Override double perimeter() { return 2 * Math.PI * radius; }
    // printInfo() inherited from Shape - no need to rewrite!
}

// Usage
Shape s = new Circle("Red", 5); s.printInfo(); // Red | area=78.54 | perim=31.42
// Shape s2 = new Shape("Blue"); // ❌ compile error - cannot instantiate abstract class
```

```
// abstract class - blueprint, not usable directly
public abstract class Shape {
    String color;

    Shape(String color) { this.color = color; }

    // Abstract method - NO body, child must implement
    abstract double area();
    abstract double perimeter();

    // Concrete method - shared logic, usable by all children
    void printInfo() {
        System.out.printf("%s | area=%.2f | perim=%.2f%n",
                           color, area(), perimeter());
    }
}
```



Use abstract classes when related classes share code AND you want to enforce a common method contract.

05 Interfaces — Pure Contracts



What is an Interface?

An interface is a 100% abstract blueprint. It defines WHAT a class can do, not HOW. A class can implement multiple interfaces (solving Java's single-inheritance limit).

Abstract Class vs Interface

	Abstract Class	Interface
Fields	Any type	public static final
Methods	Abstract + concrete	Abstract + default
Constructor	Yes	No
Extends/Implements	extends (one)	implements (many)
Use when	Shared code	Shared contract

✓ Rule of thumb: Use an abstract class when classes share CODE; use an interface when unrelated classes share BEHAVIOUR (capability).

```
// Interface definition
public interface Flyable {
    void fly();    // abstract by default
    default void land() {
        System.out.println("Landing...");
    }
}

public interface Swimmable {
    void swim();
}

// Class implements MULTIPLE interfaces!
public class Duck extends Animal
    implements Flyable, Swimmable {

    Duck(String name) { super(name); }

    @Override public void fly() {
        System.out.println(name + " is flying");
    }
    @Override public void swim() {
        System.out.println(name + " is swimming");
    }
    // land() inherited from Flyable default!
}

// Usage
Flyable f = new Duck("Donald");
f.fly(); // Donald is flying
f.land(); // Landing...
```



Exercise 3 — Abstract Classes & Interfaces



Task: Payment System

- 1 Create interface Payable with: `void pay(double amount); String getReceipt();`
- 2 Create abstract class PaymentMethod — has `String ownerName`, `double balance`. Constructor sets both. Has abstract method `validate()`. Has concrete method `printBalance()`.
- 3 Create CreditCard extends PaymentMethod implements Payable — adds `double creditLimit`. Override `validate()`: reject if `amount > creditLimit`. Implement `pay()` and `getReceipt()`.
- 4 Create BankTransfer extends PaymentMethod implements Payable — adds `String accountNo`. Override `validate()`: reject if `amount > balance`. Implement `pay()` (deduct from balance) and `getReceipt()`.
- 5 In main, create `Payable[] payments = {card, transfer};` and loop calling `pay(500)` on each.
- 6 **EDGE CASE:** Try `card.pay(9999999)` and `transfer.pay(99999)` — validate should block both.

► Expected Output

```
Paying 500.0 via CreditCard...
Receipt: Alice | CC | $500.0
Paying 500.0 via BankTransfer...
Receipt: Bob | BANK | $500.0 | Bal: $1500.0
```

-- Edge cases --

```
✗ Exceeds credit limit: 999999.0
✗ Insufficient balance: 99999.0
```



Structure Reminder

```
interface Payable {
    void pay(double amt);
    String getReceipt();
}

abstract class PaymentMethod {
    abstract boolean validate(double amt);
    void printBalance() { ... }
}

class CreditCard
    extends PaymentMethod
    implements Payable { ... }
```



Time: 20 minutes



Quick Quiz — All 5 Topics

Q1: Can you instantiate an abstract class directly?

- A) Yes, always
- B) Yes, if it has a constructor
- C) No — compile error**
- D) Only in the same package

→ *abstract classes cannot be instantiated. They exist to be extended.*

Q2: A class implements two interfaces that both define the same default method. What happens?

- A) Compiler picks one randomly
- B) Compile error — must override**
- C) Runtime error
- D) Works fine, last one wins

→ *Ambiguity in default methods is a compile error; the class must override the method.*

Q3: What keyword lets a child override a method from the parent?

- A) override
- B) @Override**
- C) virtual
- D) super

→ *@Override is an annotation, not a keyword, but it is the correct answer. It is optional but strongly recommended.*

Q4: `Animal a = new Dog(); a.makeSound();` — which `makeSound()` runs?

- A) Animal's version
- B) Dog's version**
- C) Both run
- D) Compile error

→ *Runtime polymorphism: Java dispatches to the actual object type (Dog), not the variable type (Animal).*



Mini Project

Animal Kingdom Simulator

Design a full OOP hierarchy using all 5 session topics

Inheritance

Polymorphism

Abstract Classes

Interfaces

super / @Override



Mini Project — Class Design

interface Trainable

- + train(String trick) : void
- + getSkillLevel() : int

class Cat extends Animal

- - boolean isIndoor
- + makeSound(), move()
- + scratch() : void

abstract class Animal

- # String name
- # int age
- # static int count
- Animal(name, age)
- + abstract makeSound() : void
- + abstract move() : void
- + printInfo() : void
- + static getCount() : int

class Parrot extends Animal implements Trainable

- - String[] vocabulary
- - int skillLevel
- + makeSound(), move()
- + train(word), getSkillLevel()
- + speak() : void

class Dog extends Animal implements Trainable

- - String breed
- - int skillLevel
- + makeSound(), move()
- + train(trick), getSkillLevel()

class Zoo

- - Animal[] animals
- + addAnimal(Animal a)
- + runAllAnimals()
- + trainAll() — Trainable check
- + printStats()



Mini Project — Method Logic

makeSound() in Dog / Cat / Parrot

Each subclass overrides this abstract method with its unique sound. Dog → "Woof!", Cat → "Meow!", Parrot → speaks a random word from its vocabulary array.

train(String trick) in Dog & Parrot

Validate trick is not null/empty. Increment skillLevel. Print a confirmation. Implements Trainable interface.

Zoo.trainAll()

Loop through animals. Use instanceof to check if the animal implements Trainable. If so, cast and call train("basic command").

move() in each subclass

Dog → "running", Cat → "prowling", Parrot → "flying". Each override prints name + movement style.

Zoo.runAllAnimals()

Loop through all animals, call makeSound() and move() polymorphically. Uses Animal[] — works for any future subclass.

printInfo() — inherited from Animal

Concrete method in Animal — prints name, age, class name (getClass().getSimpleName()). All children get it free.



Mini Project — Main.java & Expected Output

```
// Main.java
public class Main {
    public static void main(String[] args) {
        Zoo zoo = new Zoo(10);

        Dog rex = new Dog("Rex", 3, "Labrador");
        Cat luna = new Cat("Luna", 4, true);
        Parrot po = new Parrot("Polly", 2,
                               new String[]{"Hello", "Squawk"});

        zoo.addAnimal(rex);
        zoo.addAnimal(luna);
        zoo.addAnimal(po);

        System.out.println("=== All Animals ===");
        zoo.runAllAnimals();

        System.out.println("=== Training ===");
        zoo.trainAll();

        System.out.println("=== Stats ===");
        zoo.printStats();

        // Edge cases
        rex.train("");           // ❌ empty trick
        po.train(null);          // ❌ null trick
        zoo.addAnimal(null);      // ❌ null animal
    }
}
```

► Expected Output

```
=== All Animals ===
Rex says: Woof!
Rex is running
Luna says: Meow!
Luna is prowling
Polly says: Squawk
Polly is flying

=== Training ===
Rex learned: basic command (skill: 1)
Polly learned: basic command (skill: 1)
(Luna skipped - not Trainable)

=== Stats ===
Total animals: 3
Animal(Dog) Rex, age 3
Animal(Cat) Luna, age 4
Animal(Parrot) Polly, age 2

=== Edge Cases ===
❌ Trick cannot be empty
❌ Trick cannot be null
❌ Cannot add null animal
```




Mini Project — Starter Scaffold



Fill in the TODO sections. Do NOT change method signatures or field names.

```
public interface Trainable {
    void train(String trick);    // TODO: validate, increment skillLevel, print
    int getSkillLevel();
}

public abstract class Animal {
    protected String name; protected int age; private static int count = 0;
    Animal(String name, int age) { /* TODO */ }
    abstract void makeSound();
    abstract void move();
    void printInfo() { /* TODO - use getClass().getSimpleName() */ }
    public static int getCount() { return count; }
}

public class Dog extends Animal implements Trainable {
    private String breed; private int skillLevel = 0;
    Dog(String name, int age, String breed) { /* TODO */ }
    @Override void makeSound() { /* TODO */ }
    @Override void move() { /* TODO */ }
    @Override public void train(String trick) { /* TODO - validate first */ }
    @Override public int getSkillLevel() { return skillLevel; }
}

// Cat and Parrot follow the same pattern - implement on your own!

public class Zoo {
    private Animal[] animals; private int size = 0;
    Zoo(int capacity) { animals = new Animal[capacity]; }
    void addAnimal(Animal a) { /* TODO - validate null, check capacity */ }
    void runAllAnimals() { /* TODO - loop, call makeSound() + move() */ }
    void trainAll() { /* TODO - instanceof Trainable check */ }
    void printStats() { /* TODO */ }
}
```



Common Mistakes — Inheritance & Polymorphism



Forgetting @Override

✗ `void MakeSound() { } // typo — new method, not override!`

✓ `@Override`
`void makeSound() { } // compiler catches typo`

i Without `@Override`, a typo creates a NEW method — the parent's version runs instead.



Missing super() in child constructor

✗ `Dog(String name) {`
`this.name = name; // ✗ name is private in Animal!`

✓ `Dog(String name) {`
`super(name); // ✓ delegates to Animal's constructor`

i Private fields must be set via the parent's constructor.



Casting without instanceof check

✗ `Animal a = new Cat();`
`Dog d = (Dog) a; // ✗ ClassCastException at runtime!`

✓ `if (a instanceof Dog) {`
`Dog d = (Dog) a; // ✓ safe cast`

i Always guard downcasts with `instanceof`.



Instantiating an abstract class

✗ `Animal a = new Animal("X"); // ✗ compile error`

✓ `Animal a = new Dog("X", 3, "Poodle"); // ✓`

i Abstract classes can only be used as a reference type, not created directly.



Session 3 Cheat Sheet

extends

Lets a child class inherit from a parent class.

```
class Dog extends  
Animal { }
```

@Override

Marks a method as intentionally overriding a parent method.

```
@Override void  
makeSound() { }
```

super ()

Calls the parent constructor. Must be the first line.

```
super (name, age);
```

super.method ()

Calls the parent's version of an overridden method.

```
super.describe();
```

Polymorphism

A parent-type variable can hold a child object; runtime dispatch picks the right method.

```
Animal a = new Dog();
```

instanceof

Checks if an object is an instance of a class at runtime.

```
if (a instanceof Dog d)  
{ }
```

abstract class

Cannot be instantiated. Mixes abstract and concrete methods.

```
abstract class Shape  
{ }
```

abstract method

Has no body; child MUST override it.

```
abstract double area();
```

interface

Pure contract — all methods abstract (plus optional default methods).

```
interface Flyable  
{ void fly(); }
```

implements

A class fulfils an interface contract. Multiple allowed.

```
class Duck implements  
Flyable, Swimmable
```



Session 3 Complete!

You've mastered the pillars of OOP — inheritance, polymorphism, and abstraction.



Session 4 Preview

- Exception Handling — try / catch / finally
- Custom Exceptions
- Collections — ArrayList, HashMap
- Generics basics
- Iterator pattern



Homework

- 1 Complete the Animal Kingdom Simulator mini project if unfinished
- 2 Add a Bird abstract class between Animal and Parrot — introduce wingSpan field
- 3 Create a Swimmable interface and make both Duck and Fish implement it
- 4 Bonus: add a static leaderboard to Zoo that prints the most-trained animal

Outstanding work — you're building real OOP systems now! ☕